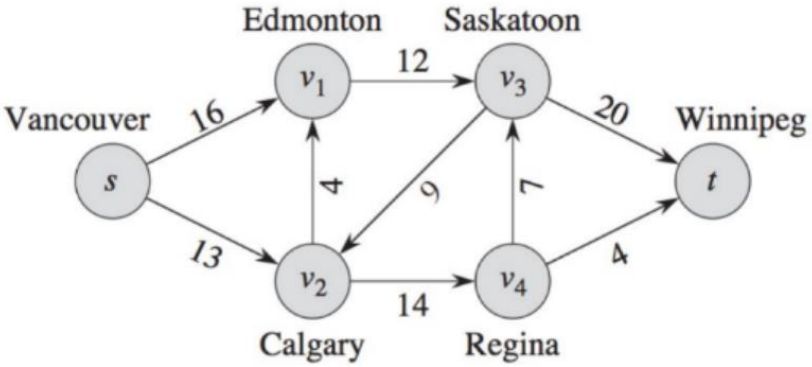
UNIVERSIDAD TECNOLÓGICA DE PEREIRA
FACULTAD DE INGENIERÍAS
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
PROGRAMACIÓN III

Actividad:

La actividad tiene como objetivo comprender y aplicar una "propuesta" para resolver un problema histórico como lo es el Problema de los Puentes de Konigsberg que pregunta si es posible dar un paseo por la ciudad cruzando cada uno de los siete puentes exactamente una vez y volviendo al punto de inicio. Esto con la finalidad de entender cómo funciona el Backtracking y los Árboles SDL en la programación lógica, específicamente al agrupar los hechos en una única estructura que este compuesta por términos **conexiones()** y haciendo uso de listas.

Para ello, se propone una versión más simple del problema mediante grafos dirigidos:



A partir de esta estructura se plantean unas preguntas (Queries) que nos permitirán construir las reglas y hechos en PROLOG:

- ¿Existe una **conexión** entre Saskatoon y Vancouver?
- ¿Con qué nodos está **conectado** Regina y cuál es el costo de cada conexión?
- ¿Un nodo X tiene aristas?
- ¿Cuál es el costo para ir de un nodo X a un Z pasando por Y?
- ¿Es posible **viajar** desde Edmonton a Calgary?

De acuerdo con estas reglas construiremos la base de conocimiento de nuestro programa junto con los hechos y reglas que harán que podamos resolver estos problemas. Cada hecho define una relación directa entre dos ciudades junto con un costo asociado, bajo la palabra clave **conexión** que es mencionada en los problemas descritos anteriormente. Este tercer parámetro (costo) actúa como una magnitud o métrica, permitiendo representar no solo "tarifas" de traslado, sino cualquier otra referencia cuantitativa (por ejemplo, distancia o tiempo) entre un punto A y un punto B.

%Hechos

A diferencia del enfoque con hechos individuales, en esta versión el grafo completo se encuentra agrupado en una única estructura principal (conexiones) que contiene una lista del tipo conexion(Origen, Destino, Costo):

```
conexiones([
conexion(vancouver, edmonton, 16),
conexion(vancouver, calgary, 13),
conexion(edmonton, saskatoon, 12),
conexion(saskatoon, winnipeg, 20),
conexion(calgary, edmonton, 4),
conexion(saskatoon, calgary, 9),
conexion(calgary, regina, 14),
conexion(regina, saskatoon, 7),
conexion(regina, winnipeg, 4)
]).
```

A partir de esta estructura basada en listas y términos compuestos, se construyen las reglas de "navegación" y cálculo de costos, analizando tanto su versión mediante recorrido explícito de listas como su versión optimizada con predicados nativos de PROLOG (member). Cabe resaltar que esta estructura se puede abarcar de formas distintas, por ejemplo:

```
conexiones([ [conexion(vancouver, edmonton), 16], ...]
```

Donde la lista está compuesta por otras listas que contienen la conexion y el costo por separado.

%Reglas

- La regla **tiene_aristas(X)** determina si un nodo X posee al menos una conexión saliente, independientemente de cuál sea su destino y su costo asociado. Para lograr esto, se plantean dos soluciones (tieneAristas1 y tieneAristas2) donde su principal diferencia radica en que la lista se pasa como argumento en una de las reglas y no se define estáticamente.

  ```
  estaConectado(X, [conexion(X,_,_)|_]).

  estaConectado(X, [_|Cola]) :-
          estaConectado(X, Cola).
  ```

tieneAristas1(X) :-

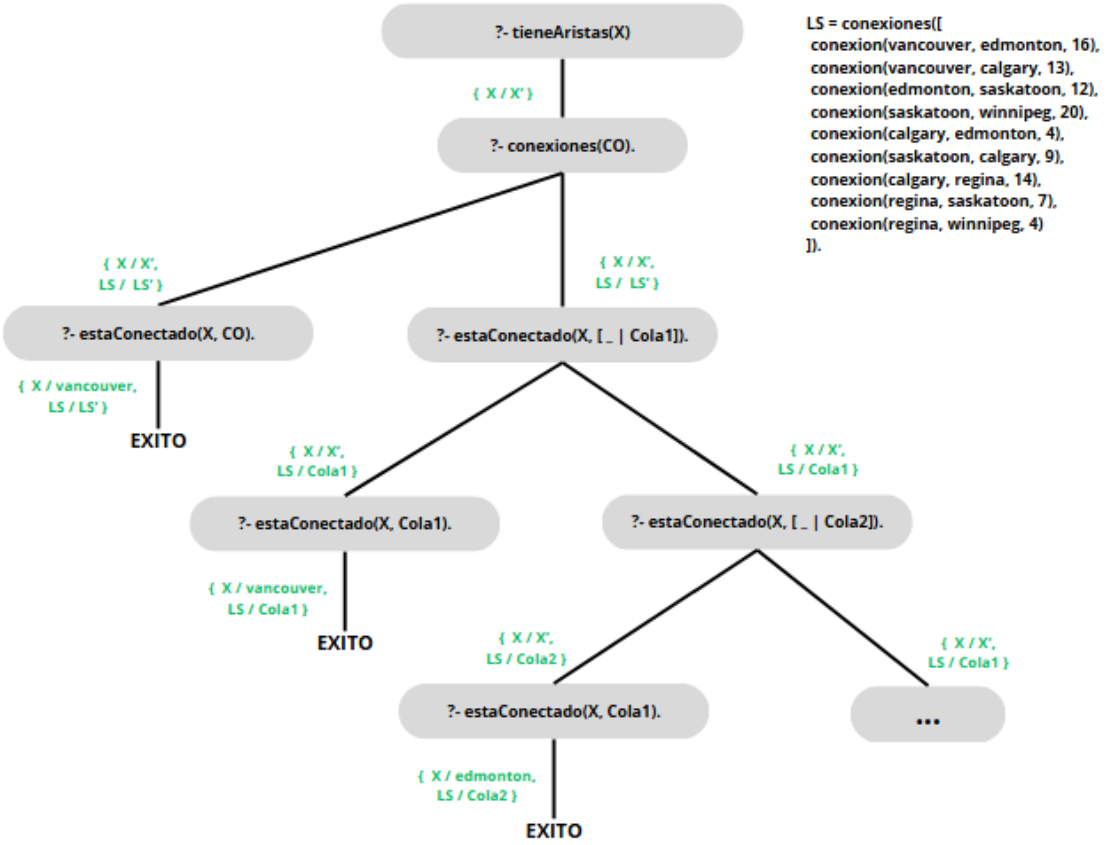
conexiones(LS),

estaConectado(X, LS).

tieneAristas2(X, CO) :-

estaConectado(X, LS).

Para comprender como funciona analizaremos esta regla mediante la consulta:
?- tieneAristas1(X)



Para determinar si un nodo particular posee al menos una arista saliente, se emplea el predicado **tieneAristas1**, cuya evaluación depende directamente de la regla auxiliar **estaConectado**. Cuando se ejecuta una consulta como **?- tieneAristas1(X).**, Prolog inicia el proceso obteniendo en primer lugar la lista completa de conexiones almacenada en el hecho **conexiones(CO)**. Posteriormente, se evalúa **estaConectado(X, CO)** para determinar la presencia del nodo en la estructura. Al resolver este predicado, Prolog intenta unificar la meta con su caso base **estaConectado(X, [conexion(X,_,_)|_]).**

La primera coincidencia ocurre en la cabeza de la lista con la estructura **conexion(vancouver, edmonton, 16)**, lo que permite unificar la variable asignándole el valor vancouver y devolviendo una primera respuesta exitosa. Si se solicita una nueva solución mediante el operador de backtracking **(;)**, Prolog

retrocede en el Árbol SLD para buscar alternativas. Al no existir más opciones en la cabeza del caso base para ese primer elemento, el sistema recurre al caso recursivo **estaConectado(X, [_|Cola])**, el cual "elimina" el primer elemento de la lista y realiza una nueva llamada pasando únicamente el resto de la estructura (Cola). Este proceso recursivo de rebanado de listas se repite sucesivamente, explorando la lista elemento por elemento y unificando la variable X con los orígenes de cada conexión registrada (edmonton, saskatoon, calgary, regina) hasta recorrer la totalidad de la lista, momento en el cual no se logran más unificaciones y el backtracking finaliza retornando false.

- A diferencia de la regla anterior, esta regla permite determinar el alcance entre dos nodos X e Y sin importar cuántos nodos intermedios existan, gracias al uso combinado de recursividad y backtracking. Esta lógica puede implementarse tanto mediante la utilización del predicado explícito member **(viajar1)** como mediante el manejo manual de listas **(viajar2).**

  Esta lógica se divide obligatoriamente en dos cláusulas independientes (que actúan como una disyunción lógica o 'OR'):

  - **El caso base:** Verifica si existe una conexión directa.
  - **El caso recursivo:** Avanza a través de un nodo intermedio Z y se llama a sí mismo para continuar la ruta.

  Definirlas por separado es indispensable. Si se intentara unificar ambas condiciones en una sola regla mediante una conjunción (operador **","**), se exigiría que toda búsqueda cumpla simultáneamente con una conexión directa y un tramo recursivo, lo cual invalidaría toda la regla para trayectos largos.

```
viajar1(CIUDAD_A, CIUDAD_B, LS) :-
        member(conexion(CIUDAD_A, CIUDAD_B, _), LS).

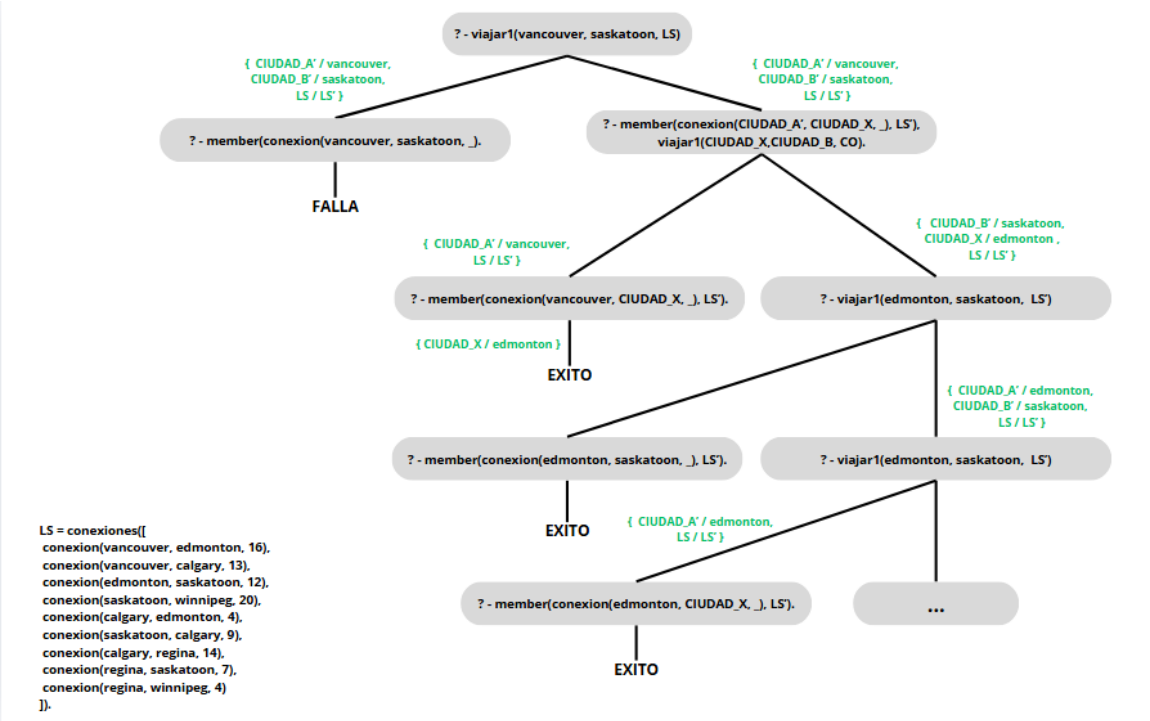
viajar1(CIUDAD_A, CIUDAD_B, LS) :-
        member(conexion(CIUDAD_A, CIUDAD_X, _), LS),
        viajar2(CIUDAD_X, CIUDAD_B, LS).

viajar2(CIUDAD_A,CIUDAD_B, [conexion(CIUDAD_A, CIUDAD_B, _)|_]).

viajar2(CIUDAD_A, CIUDAD_B, [_ | Cola]) :-
        viajar1(CIUDAD_A, CIUDAD_B, Cola).
```

viajar2(CIUDAD_A, CIUDAD_B, [conexion(CIUDAD_A, CIUDAD_X, _)|_]) :-
LS = [conexion(CIUDAD_A, CIUDAD_X, _)|_],
viajar1(CIUDAD_X, CIUDAD_B, LS).

Para evaluar alguna de las reglas**,** Prolog combina el Árbol de Resolución SLD con backtracking a través de sus dos cláusulas (caso base y caso recursivo):



En la versión **viajar1**, una consulta como **?- viajar1(CIUDAD_A, CIUDAD_B, LS)**, Prolog evalúa inicialmente el caso base mediante **member(conexion(CIUDAD_A, CIUDAD_B, _), LS)**, buscando si existe una conexión directa registrada entre ambas ciudades dentro de la lista. En caso de no existir un enlace directo, el caso base falla en esa rama del Árbol SLD, lo que desencadena un backtracking inmediato hacia el caso recursivo. En esta segunda cláusula, Prolog busca un salto intermedio mediante la **member(conexion(CIUDAD_A, CIUDAD_X, _), LS)**, unificando **CIUDAD_X** con la primera ciudad alcanzable.

Inmediatamente después, genera una llamada recursiva con **viajar1(CIUDAD_X, CIUDAD_B, LS)**. En este nuevo nivel de recursión, se evalúa en primer lugar el caso base para determinar si desde la nueva ciudad intermedia se puede llegar directamente al destino final. De no ser así, el proceso continúa profundizando recursivamente en el grafo hasta encontrar una secuencia de saltos que unifique con el caso base o hasta agotar todas las combinaciones posibles en la lista.

Y para la versión **viajar2** la exploración de la lista se realiza de forma explícita manejando tres cláusulas. La primera actúa como el caso base de conexión directa en la cabeza de la lista **viajar2(CIUDAD_A, CIUDAD_B, [conexion(CIUDAD_A, CIUDAD_B, _)|_])**, la segunda permite avanzar iterativamente sobre la estructura descartando elementos no coincidentes mediante **viajar2(CIUDAD_A, CIUDAD_B, [_ | Cola])**, y la tercera representa el paso recursivo en el grafo:

```
viajar2(CIUDAD_A, CIUDAD_B, [conexion(CIUDAD_A, CIUDAD_X,_)|_]) :-
        CO = [conexion(CIUDAD_A, CIUDAD_X, _)|_],
        viajar2(CIUDAD_X, CIUDAD_B, CO).
```

Ante una consulta como **?- viajar2(CIUDAD_A, CIUDAD_B, CO)**, Prolog evalúa inicialmente si el elemento actual en la cabeza de la lista conecta directamente ambas ciudades. Si la cabeza no coincide en destino o en origen, la regla de avance descarta dicho elemento pasando la cola restante para continuar la búsqueda dentro de la lista. Cuando la cabeza contiene una salida desde **CIUDAD_A** hacia un nodo intermedio **CIUDAD_X**, se unifica la variable intermedia y la tercera cláusula genera una llamada recursiva **viajar2(CIUDAD_X, CIUDAD_B, CO)** reevaluando nuevamente toda la lista de conexiones para hallar el siguiente tramo.

En ambos enfoques, si una rama del Árbol SLD no conduce al destino, el backtracking retrocede para explorar conexiones alternativas hasta agotar las combinaciones posibles en la lista.

- La regla **viajarCosto1(X, Y, COSTO, LS)** o **viajarCosto2(X, Y, COSTO, LS)** extienden la lógica de alcance del predicado **viajar{i}(X, Y)** al añadir la suma de costos de las aristas del grafo mediante el operador **“is”**. Mientras que el primero solo retorna un valor de verdad (verdadero o falso), esta regla calcula y devuelve el costo total de la ruta. Su funcionamiento opera de forma recursiva:

  - **Caso base:** Retorna directamente el costo de una conexión directa.
  - **Caso recursivo:** Desglosa el trayecto en un tramo inicial (COSTO1) y el resto del camino (COSTO2), sumándolos mediante el operador aritmético is.

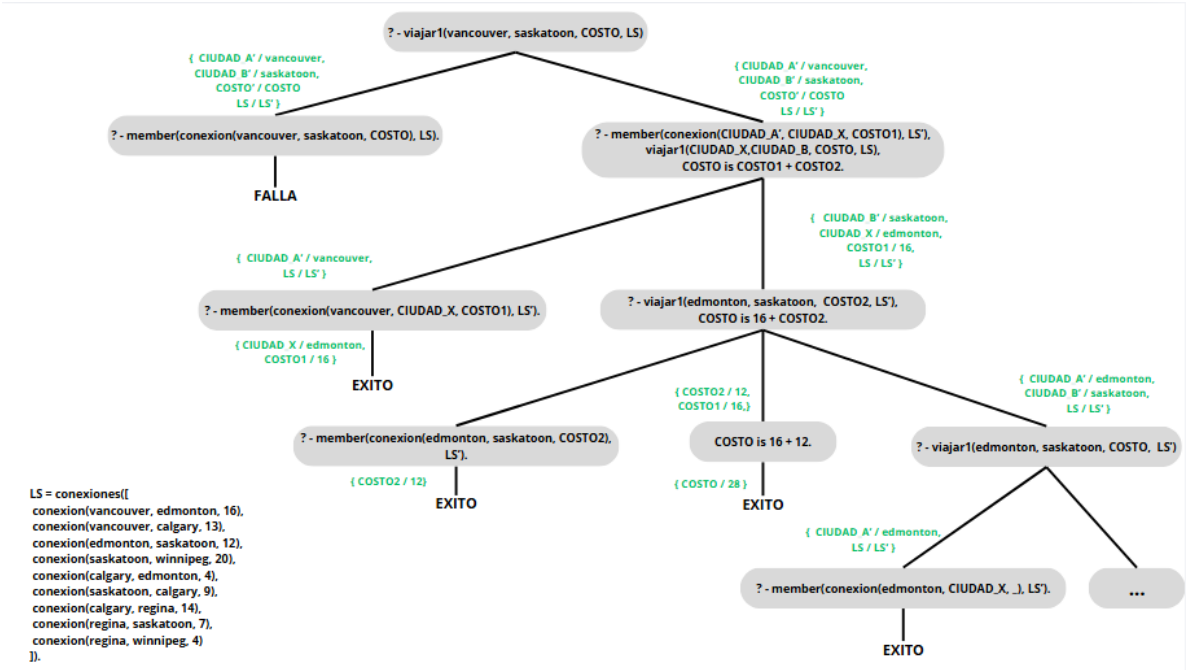
```
viajarCosto1(CIUDAD_A, CIUDAD_B, COSTO, LS) :-
        member(conexion(CIUDAD_A, CIUDAD_B, COSTO), LS).
```

viajarCosto1(CIUDAD_A, CIUDAD_B, COSTO, LS) :-
member(conexion(CIUDAD_A, CIUDAD_X, COSTO1), LS),
viajarCosto1(CIUDAD_X, CIUDAD_B, COSTO2, LS),
COSTO is COSTO1 + COSTO2.

viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, [conexion(CIUDAD_A, CIUDAD_B, COSTO) |_]).

viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, [_ | Cola]) :-
viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, Cola).

viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, LS) :-
LS = [conexion(CIUDAD_A, CIUDAD_X, COSTO1)|_],
viajarCosto2(CIUDAD_X, CIUDAD_B, COSTO2, LS),
COSTO is COSTO1 + COSTO2.



Para la versión **viajarCosto1** al evaluar una consulta de costo como **?-viajarCosto1(CIUDAD_A, CIUDAD_B, COSTO, LS)**, el sistema intenta primero resolver la trayectoria mediante una conexión directa en el caso base. Al no encontrar un enlace directo entre dos puntos distantes, la rama del caso base falla y el sistema evalúa el caso recursivo.

En este punto, **member(conexion(CIUDAD_A, CIUDAD_X, COSTO1), LS)** extrae el costo del primer tramo directo **(COSTO1)**, dejando pendiente la evaluación de la suma aritmética **COSTO is COSTO1 + COSTO2** mientras genera una llamada recursiva **viajarCosto1(CIUDAD_X, CIUDAD_B, COSTO2, LS)** para calcular el

costo del resto del camino. Conforme el algoritmo profundiza recursivamente en el Árbol SLD, las operaciones aritméticas quedan suspendidas en la pila de evaluación hasta que la última llamada recursiva alcanza el caso base y retorna un valor de costo directo. En ese momento, Prolog resuelve las operaciones **“is”** pendientes de abajo hacia arriba, sumando secuencialmente los costos parciales acumulados en cada sub-ruta hasta asignar el costo total final a la variable COSTO.

Al forzar el backtracking, se deshacen las unificaciones previas y explora caminos alternativos dentro de la lista, evaluando nuevas combinaciones de saltos intermediarios y recalculando sus respectivos costos hasta agotar todas las alternativas representadas en el grafo.

Y para la versión **viajarCosto2** se combina la búsqueda manual dentro de la lista con la acumulación del costo a lo largo del camino. La primera cláusula verifica si la cabeza de la lista contiene una conexión directa **viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, [conexion(CIUDAD_A,CIUDAD_B,COSTO)|_])**, la segunda cláusula **viajarCosto2(CIUDAD_A, CIUDAD_B, COSTO, [_ | Cola])** permite recorrer la cola de la lista cuando la cabeza no coincide, y la tercera cláusula gestiona los saltos indirectos evaluando la coincidencia del origen **CO = [conexion(CIUDAD_A, CIUDAD_X, COSTO1)|_]**, invocando el caso recursivo **viajarCosto2(CIUDAD_X, CIUDAD_B, COSTO2, CO)** y realizando la suma final **COSTO is COSTO1 + COSTO2**